

EGYSPY

Review of Real-Time Tracking and Streaming Technologies

Task 7 — Literature Review Deliverable

Prepared for the EgySpy pre-implementation phase
Draft v1.0 | 10 October 2026

Research question	Which technologies transport and process location data in real time, and how do they compare on latency, scalability and complexity?
Project context	Employee movement tracking within a defined facility; capture visited zones and time spent, and support management monitoring.
Scope boundary	Technology comparison only. No hardware selection, implementation or prototype is decided by this document.
Evidence approach	Official standards and vendor/project documentation, supplemented by experimental and peer-reviewed research. Performance claims are context-dependent.

Executive Summary

EgySpy needs a reliable path from a location system or hardware gateway to application services and then to a live management dashboard. The technologies considered here do not all solve the same problem: MQTT and WebSocket are communication options; Apache Kafka is a durable distributed event-streaming platform; Kafka Streams and Apache Flink are stream-processing options; and edge computing is a deployment approach that moves some processing closer to the data source.

The most practical initial candidate is a small, separable pipeline: an RTLS system or gateway exposes location events through a supported interface (preferably MQTT or a documented API); a backend service validates, timestamps and stores events; and WebSocket delivers live updates to the browser dashboard. This is a candidate for evaluation, not a final architecture decision. Kafka, Kafka Streams or Flink should be considered if measured event volume, replay requirements, multiple independent consumers, complex event-time logic or availability requirements justify their operational cost.

Hardware integration is likely to be easiest when the selected RTLS vendor already exposes normalized location events through MQTT/REST APIs. Integrating directly with a raw UWB module is a different and more hardware-intensive task: it may require firmware/SDK work, radio configuration, anchor/tag setup, calibration, and a positioning engine or gateway. Public vendor documentation confirms that both patterns exist, but the exact integration effort depends on the hardware and product chosen [9], [10].

1. Project Requirements and Comparison Method

The review follows the project Literature Review Plan and its research question RQ3. The relevant EgySpy objectives are continuous movement tracking and management monitoring; the broader project also needs records of visited locations and time spent in zones. The comparison therefore focuses on timely delivery, growth in devices/events, operational complexity, and integration with the eventual RTLS hardware.

Evaluation criteria used in this document:

- **Latency:** how quickly an event can travel from a device/gateway through processing to the dashboard. No universal latency number is assigned unless a cited experiment provides it.
- **Scalability:** ability to support more tags, higher update frequency, multiple consumers and more processing work.
- **Complexity:** deployment, configuration, monitoring, security, fault handling and maintenance burden.
- **Hardware integration:** ease of connecting the technology to tags, anchors, an RTLS engine or a gateway. Ratings are qualitative and depend on available interfaces and SDKs.
- **Project fit:** how directly the option supports EgySpy's initial needs without adding unnecessary infrastructure.

Evidence caution: experimental results from one board, network, broker, payload or Quality of Service setting must not be generalized to every deployment. For example, studies comparing MQTT and WebSocket have reported different winners under different setups [6]–[8].

2. Technologies and Their Roles

2.1 MQTT — device/gateway messaging

MQTT is a lightweight publish/subscribe messaging protocol standardized by OASIS. Clients publish messages to topics through a broker; subscribers receive messages matching their subscriptions. MQTT defines delivery Quality of Service (QoS) levels: at most once, at least once, and exactly once at the protocol-delivery level [1]. QoS choices affect delivery behavior and overhead, so they should be chosen according to whether an event is a frequent location update, a critical zone-entry event, or an audit-relevant record.

Advantages: lightweight messaging; producer and consumer are decoupled; topic subscriptions can distribute location events to multiple services; established fit for constrained IoT devices and gateways [1]. Disadvantages: requires a broker

and topic/security design; QoS does not by itself guarantee that the application has persisted or acted on a location event; duplicate handling and application-level event identifiers may still be needed.

EgySpy fit: strong candidate for the device/gateway-to-backend path if the RTLS system or gateway can publish normalized location events. It is not a location technology and does not calculate the employee's position.

2.2 WebSocket — live browser updates

WebSocket provides a persistent, bidirectional connection between a client and server, allowing the server to send updates without the browser polling for each change [2]. This makes it useful for moving accepted location updates from the backend to the live dashboard.

Advantages: direct browser support; natural fit for interactive maps and live status; can support two-way messages where needed [2]. Disadvantages: the standard WebSocket API does not provide automatic backpressure; if events arrive faster than the client can process them, buffers, memory use or CPU load can become a problem [2]. The backend should therefore throttle, batch, coalesce stale position updates or otherwise manage client load.

EgySpy fit: strong candidate for backend-to-dashboard delivery. It should generally be treated as a UI delivery channel, not as a replacement for a broker or durable event log.

2.3 Apache Kafka — durable event streaming

Kafka is an event-streaming platform that can publish and subscribe to event streams, store them durably, process them, and route them to other systems. Its distributed design supports partitioning, parallel consumers and fault tolerance [3].

Advantages: durable event history and replay; multiple independent consumers; partition-based scaling; useful when location events feed live views, audit/history storage, analytics and alerts concurrently [3]. Disadvantages: introduces broker-cluster operations, partition/key design, retention configuration, monitoring and additional integration components. It is usually not a direct protocol for a small microcontroller; a gateway or service normally bridges hardware events into Kafka.

EgySpy fit: conditional. Consider it if the project demonstrates high event volume, multiple downstream consumers, replay needs or stronger separation between ingestion and processing. For an early prototype, it may be unnecessary overhead.

2.4 Kafka Streams — application-level processing over Kafka

Kafka Streams is a client library for building stream-processing applications on top of Kafka. It uses topic partitions and processing tasks to support parallelism and scaling [4].

Advantages: useful for transformations, aggregations, joins and stateful processing when Kafka is already part of the design; fewer separate platform components than introducing a different processing engine [4]. Disadvantages: depends on Kafka; requires developers to understand stream semantics, state, partitioning and recovery. It is not needed merely to forward a current position to a dashboard.

EgySpy fit: a later option for zone-duration aggregation, event correlation or deriving movement summaries if Kafka is selected and the processing requirements justify it.

2.5 Apache Flink — advanced stream processing

Apache Flink is a distributed engine for stateful computations over bounded and unbounded data streams. Its documented capabilities include event-time processing, state management, checkpointing and recovery [5].

Advantages: suited to more complex continuous processing, event-time windows, late/out-of-order data and substantial stateful computations [5]. Disadvantages: additional deployment and operational complexity; needs careful treatment of state, checkpoints, event time, recovery and data sinks. It may be excessive for a simple first version.

EgySpy fit: consider if future requirements need sophisticated stream logic—such as robust duration calculations across late events, complex zone transitions or large-scale continuous analytics—and only after those requirements are established.

2.6 Edge computing — deployment pattern, not a messaging protocol

Edge computing places some computation near the device or local network instead of sending every raw reading to a remote central service. For EgySpy, an edge gateway could normalize vendor-specific payloads, buffer briefly during connectivity loss, or filter noisy/redundant updates before forwarding them.

Advantages: may reduce upstream traffic and support local reactions where network conditions require them.

Disadvantages: creates distributed devices to secure, update, monitor and recover; behavior must remain consistent between the edge and central services. The benefit is workload- and network-dependent, so it should be justified by a measured need rather than assumed.

3. Comparative Matrix

Option	Primary role	Latency	Scalability	Complexity	Hardware integration	EgySpy fit
MQTT	Device/gateway → backend messaging	Low-latency messaging is feasible; actual latency depends on network, broker, QoS and payload.	Good pub/sub decoupling; broker capacity and topic design matter.	Low–medium	High when the RTLS gateway/device supports MQTT; otherwise a bridge is needed.	Strong initial candidate for location-event ingestion [1], [6], [7], [9].
WebSocket	Backend ↔ browser live updates	Interactive updates without polling; not a universal fastest protocol. Client backpressure must be managed.	Persistent connections scale with server/client capacity and connection management.	Low–medium	High for web clients; not the usual direct interface for raw RTLS hardware.	Strong candidate for dashboard updates [2], [6]–[8].
Apache Kafka	Durable event log and multi-consumer distribution	Can support high-throughput streaming; end-to-end latency depends on configuration and consumers.	High via partitions/cluster, but needs operational planning.	High	Low direct hardware integration; usually needs gateway/service/connector.	Conditional: use when replay, multiple consumers or load justify it [3].
Kafka Streams	Processing events already in Kafka	Depends on processing topology, state and workload.	Scales through partitions and processing tasks.	Medium–high	Indirect; consumes Kafka records rather than hardware signals.	Useful later for event aggregation/correlation if Kafka is adopted [4].
Apache Flink	Distributed stateful stream processing	Designed for low-latency stream computation, but actual latency is workload/configuration-dependent.	High scale-out capability with significant operational requirements.	High	Indirect; requires a source connector or service/gateway feeding streams.	Advanced option, not required for basic live tracking [5].
Edge computing	Local processing near RTLS hardware	Can reduce network round trips for selected tasks; benefit must be measured.	Distributed across gateways/devices; fleet management becomes important.	Medium–high	Potentially high when hardware exposes local interfaces, but firmware and device constraints vary.	Use only for a concrete local-processing, connectivity or bandwidth requirement.

4. Hardware Integration Comparison

The main integration variable is not the messaging tool alone; it is what the selected RTLS hardware and location engine expose. A managed RTLS product may already provide location events through MQTT or REST, whereas a raw UWB development kit may require embedded firmware, ranging configuration, anchor/tag setup, a positioning algorithm and a gateway.

Integration scenario	Ease (qualitative)	Typical integration path	Main risks / caveats	Evidence / rationale
RTLS platform with documented MQTT stream/API	High / easiest	Subscribe to the location stream or call the documented API; map the vendor payload to an internal event schema.	Vendor-specific payload and licensing/access constraints; confirm whether people tags and zone events are supported.	Pozyx documents MQTT and REST/API integration options [9].
UWB development kit / raw module	Low–medium / most work	Use vendor SDK/firmware and hardware interfaces; build or connect to an RTLS positioning engine; then expose normalized events to MQTT/HTTP.	Radio/firmware skills, anchor geometry and calibration, power/battery, radio environment and driver support.	Qorvo DWM3001CDK exposes USB, SWD/UART and Raspberry Pi-compatible interfacing for evaluation [10].
Gateway between RTLS and backend	Medium	Read the vendor output locally, normalize IDs/timestamps/coordinates, buffer if needed, then publish upstream.	Gateway becomes another component to secure, monitor and maintain.	A useful integration boundary when hardware interfaces are proprietary or inconsistent.
WebSocket directly from a hardware device	Low–medium, case-dependent	Possible only if device firmware/network stack supports it	Device resource limits and connection lifecycle; browser-	Prefer WebSocket for dashboard delivery rather

		and the deployment warrants it; more commonly the backend owns WebSocket connections.	facing auth/authorization is still needed.	than assuming it is the hardware ingestion protocol [2].
Kafka directly from a small embedded tag	Low / generally not preferred	Usually bridge device data through a gateway or backend producer.	Kafka client/broker architecture is heavier than a small tag needs; network/security and resource constraints.	Use Kafka at the server side if justified, not as the default firmware protocol [3].

Interpretation: “High” and “Low–medium” are qualitative engineering judgments based on the documented interfaces and component roles, not measured integration-time benchmarks. The team should revisit these ratings after choosing a candidate RTLS kit and confirming its software/API access.

5. What Experimental Research Says About MQTT vs WebSocket

Published experiments show why a universal winner should not be claimed. Mijovic, Shehu and Buratti compared CoAP, WebSocket and MQTT on a low-cost IoT hardware platform and measured protocol overhead and round-trip time under LAN and Internet scenarios [6]. Ajayi et al. compared MQTT publish/subscribe with WebSocket client/server streaming for IoT telemetry, examining throughput, round-trip time and resource utilization; their reported results favored MQTT for throughput/resource use, while WebSocket was slightly faster in round-trip time in the tested setup [7]. A later ESP32-S3 study reported MQTT with QoS 0 outperforming WebSocket in its local-LAN experiment for mean latency, jitter and packet-loss rate [8].

These findings are not contradictory in a way that permits one blanket ranking: the hardware, broker placement, network, payloads, sample counts, implementation and QoS differ. For EgySpy, a small proof-of-concept benchmark should measure end-to-end event age (sensor/gateway timestamp to dashboard rendering), message loss/duplicates, throughput at expected tag update rates, CPU/memory and recovery after disconnects.

6. Proposed Candidate Flow for EgySpy (Not a Final Architecture Decision)

1. RTLS hardware/engine: produces a normalized location event with employee/tag ID, x/y (and floor/zone if available), timestamp, confidence/quality and source ID.
2. Ingestion adapter: receives MQTT or a documented vendor API; validates schema, authenticates the source, deduplicates events where needed and records the source timestamp.
3. Backend service: applies access rules, maps coordinates to zones, calculates zone transitions/duration and persists audit-relevant events. Avoid treating an unverified location update as a security decision.
4. Storage: persists event history and current/latest position separately where appropriate; retention and employee-privacy rules must be defined by the project.
5. Dashboard delivery: backend pushes relevant current-state changes to authorized clients over WebSocket. Coalesce stale positions or throttle display updates if the browser cannot keep up.
6. Scale-out trigger: introduce Kafka if durable replay, multiple independent consumers or measured throughput justify it; add Kafka Streams or Flink only when stream-processing requirements outgrow a simpler backend service.

This flow is a recommendation for evaluation, not an approved implementation. The project plan explicitly leaves technology and architecture choices open at the literature-review stage.

7. Risks, Gaps and Validation Needed

- No selected RTLS hardware/vendor is confirmed in the materials reviewed; hardware-integration ease is therefore conditional.
- No EgySpy workload target is yet established here (number of tags, update frequency, retention period, concurrent dashboard users). Without it, absolute latency or capacity claims would be speculative.
- Vendor documentation describes supported integration interfaces but is not independent evidence of comparative performance.

- MQTT QoS settings, duplicate events, timestamps, out-of-order updates and reconnect behavior must be considered in the event schema and tests.
- WebSocket browser delivery needs authentication, per-user authorization, connection limits and a strategy for slow clients.
- Employee movement data is sensitive operational data. The project should define least-privilege access, encryption in transit, retention and audit policy before implementation.
- The statement that existing systems do not combine all EgySpy goals must be validated through the broader comparative synthesis; this document does not prove that no such system exists.

8. Conclusion and Recommendation

For the current planning phase, MQTT and WebSocket are the most directly relevant communication options because they address different sides of the pipeline: MQTT is a candidate for device/gateway-to-backend messaging, while WebSocket is a candidate for backend-to-dashboard updates. The hardware integration decision should prioritize an RTLS platform or gateway with a documented, supportable event interface. If the chosen system exports MQTT or REST location events, it can reduce the need to implement a low-level UWB integration in the graduation project.

Kafka is a scale/replay option rather than an automatic requirement; Kafka Streams and Flink are processing options for more demanding event logic. The project should first define expected tag count, update frequency, latency target, number of consumers and history/retention requirements, then benchmark the smallest viable pipeline. No final product, hardware module or architecture is selected by this review.

References

- [1] OASIS, “MQTT Version 5.0,” OASIS Standard. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [2] MDN Web Docs, “WebSocket API (WebSockets).” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- [3] Apache Software Foundation, “Apache Kafka Documentation — Introduction / Event Streaming.” [Online]. Available: <https://kafka.apache.org/documentation/>
- [4] Apache Software Foundation, “Kafka Streams Architecture.” [Online]. Available: <https://kafka.apache.org/documentation/streams/architecture/>
- [5] Apache Software Foundation, “Apache Flink Documentation.” [Online]. Available: <https://flink.apache.org/q/stable-docs.html>
- [6] S. Mijovic, E. Shehu, and C. Buratti, “Comparing Application Layer Protocols for the Internet of Things via Experimentation,” in Proc. IEEE 2nd International Forum on Research and Technologies for Society and Industry (RTSI), 2016, doi: 10.1109/RTSI.2016.7740559.
- [7] O. Ajayi, A. Bagula, J. Bode, and M. Damon, “A Comparison of Publish-Subscribe and Client-Server Models for Streaming IoT Telemetry Data,” in AFRICATEK, 2023, doi: 10.1007/978-3-031-35883-8_9.
- [8] M. F. Tsaqief and J. Sutopo, “Comparative Performance Analysis Between the MQTT and WebSocket Protocols,” bit-Tech, vol. 8, no. 2, pp. 2227–2237, Dec. 2025, doi: 10.32877/bt.v8i2.3223.
- [9] Pozyx, “Pozyx Documentation — RTLS and Platform Integration.” [Online]. Available: <https://www.pozyx.io/documentation>
- [10] Qorvo, “DWM3001CDK — DWM3001C, DWM3110 Ultra-Wideband Module Development Kit.” [Online]. Available: <https://www.qorvo.com/products/ek/DWM3001CDK>
- [11] B. A. Enache, C. K. Banica, and A. G. Bogdan, “Performance Analysis of MQTT Over WebSocket for IoT Applications,” Scientific Bulletin of Electrical Engineering Faculty, vol. 23, no. 1, pp. 46–49, 2023, doi: 10.2478/sbeef-2023-0008.

Source and Scope Note

This document is a research-backed draft for Task 7, not a claim that the system has been implemented or benchmarked. The project Literature Review Plan defines the target as approximately 8–12 relevant sources per core review and requires a common source-summary format and IEEE-style references. Before submission, the team should verify bibliographic metadata against publisher records, agree on the final citation style and source log, and align terminology with the consolidated project proposal.